Module 2 **Verilog HDL**

Dr. Anand S.

Professor
Centre for Nanotechnology Research (CNR)
Vellore Institute of Technology, Vellore

Module 2 Verilog HDL

Module:2	Verilog HDL	5 hours
Lexical Conventions, Ports and Modules, Operators, Dataflow Modelling, Gate Level Modelling, Behavioural Modeling, Test Bench.		

Basics of Verilog HDL

HDL – Hardware Description Language

Two Competing HDL's

VHDL – VHSIC HDL

VHSIC – Very High Speed Integrated Circuit

Verilog HDL

Verilog HDL is most widely used HDL – IEEE industry standard HDL used to describe a digital system.

Provides comprehensive support for low level design.

Verilog was designed primarily for digital H/W designers developing FPGA's/ASICs

Terminologies

Simulation => check if design works fine

Synthesis => Conversion of Register Transfer Level (RTL) to gate level (implement the design on real hardware)

History of Verilog HDL

- Originated in 1983 @ Gateway Design Automation.
- In 1989 Cadence design systems, purchased Gateway and made Verilog language to be available to the public.
- Verilog language then became IEEE standard 1364, referred to as Verilog-95.
- Extension to Verilog-95 were introduced in 2001 referred to as Verilog - 2001 and minor revisions introduced in 2005.
- In 2009, the SystemVerilog and Verilog language standards were merged.
- The current version is IEEE standard 1800-2017

Verilog Module

- In verilog, the basic unit of hardware is called module.
- Modules cannot contain the definitions of other module
- A Module can, however, be instantiated with in another module.
- Allows the creation of a hierarchy in a Verilog description.
- Semicolon : Statement terminator
- // - Single line comment
- /* */ - Multi-line comment
- Case sensitive

Module Structure

```
module name(portlist);
    port declarations;
    parameter declarations;

    wire declarations;
    reg declarations;
    variable declarations;

    module instantations;
    dataflow statements;
    always blocks;
    initial blocks;

    tasks and functions;

endmodule
```

Example 1 : simple XOR Gate

```
module simplexor (f,a,b);
    input a, b;
    output f;
    assign f = a^b;
endmodule
```

Identifier & Keywords

Identifier

User-provided names for Verilog objects in the descriptions

Legal characters are “a-z”, “A-Z”, “0-9”, “_”, and “$”

First character has to be a letter or an “_”

Example: Count, _R2D2, FIVE$

Keywords

Predefined identifiers to define the language constructs

All keywords are defined in lower case

Cannot be used as identifiers

Example:**initial, assign, module, always….**

Ports

Three types of ports

- Input (keyword - **input**)
- Output (keyword – **output**)
- Inout (keyword – **inout**)

Port declarations example

input a;

input a, b;

input [3:0] c, d;

output [4:0] y;

inout x;

Data Types

Two basic data types

- Nets and
- Registers

Nets represent physical wires in the design.(**wire, tri**)

- Default initial value for a wire is “Z”

Registers represent storage in the Verilog model. (**reg, integer, real, time**)

- Register data type may or may not result in physical registers.
- Registers are manipulated within procedural blocks (***always*** and ***initial***) only.
- Default initial value for a reg is “X”

Data Types - Examples

```
reg  a;  // a scalar register

reg [3:0]  v;  // a 4-bit vector register from msb to lsb

reg [7:0]  m, n; // two 8-bit register m and n

tri   [15:0]  busa;  // a 16-bit tri-state bus


wire [31:0] w1, w2;

// Two 32-bit wires w1 and w2 with msb being the 31st bit
```

Register Types

reg

any size, unsigned

integer (not synthesizable)

```
integer a,b;  // declaration(a = 10, b = -10)
```

real (not synthesizable)

```
real a,b;  // declaration(a = 3.14, b = 3e6)
```

time (not synthesizable)

64-bit unsigned, behaves like a 64-bit reg

Numbers & Negative Numbers

A number may be **sized** or **unsized**

Ex: h12_unsized = ‘h12; h12_sized = 6’h12;

Ex: PA = -12, PB = -’d12, PC= -32’d12;

Sized numbers are written as **width ‘radix value**

The **radix** indicates the type of number

Decimal (d or D)

Hex (h or H)

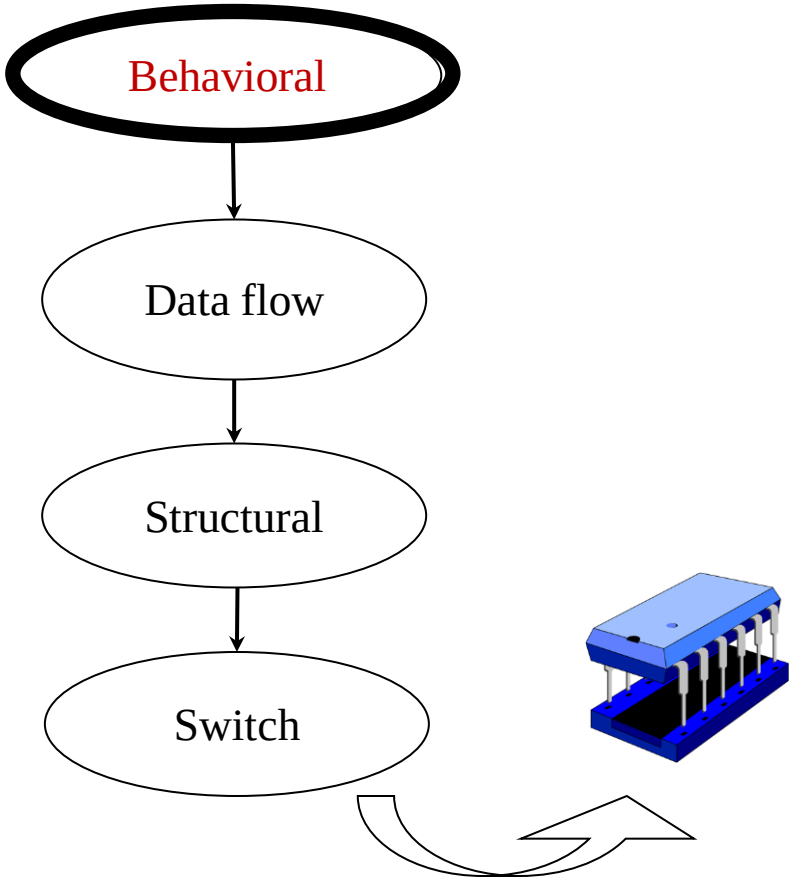
Octal (o or O)

Binary (b or B)

Unsized numbers are written as **‘radix value**

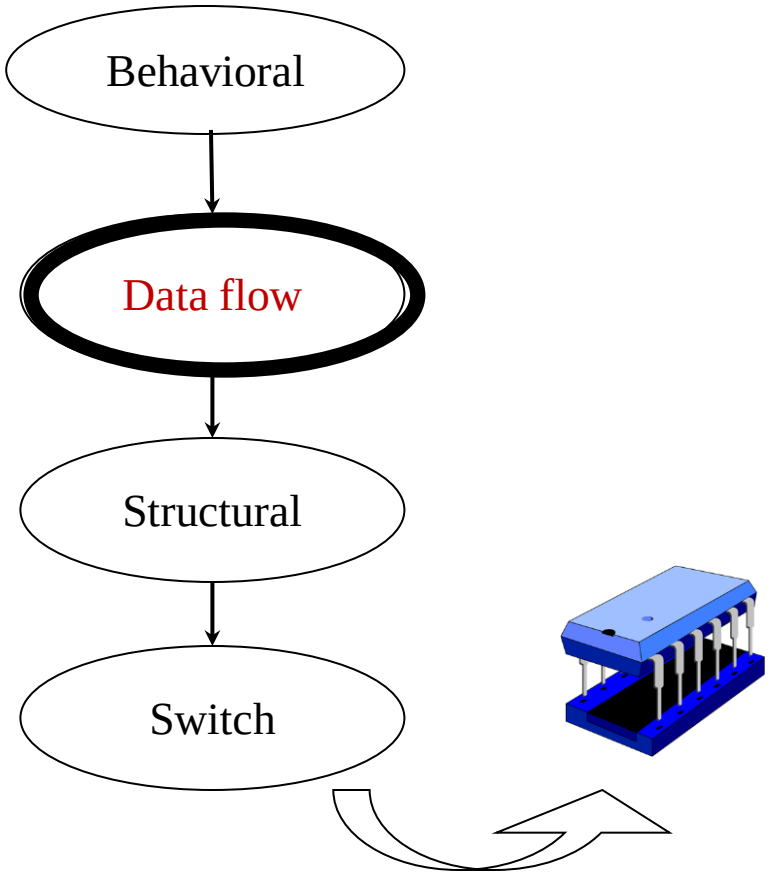
Levels of Abstraction

Module can be implemented in terms of design algorithm without concern for the hardware implementation (similar to C programing)



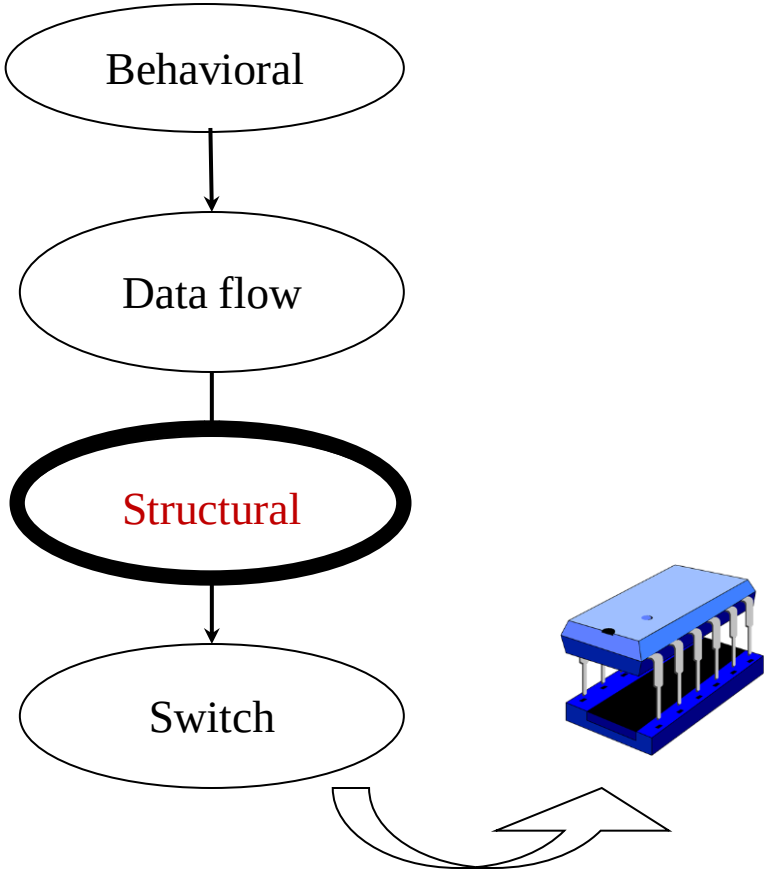
Levels of Abstraction

Module is designed by specifying the data flow. Designer is aware on how the data flows between different nodes and how it is processed.

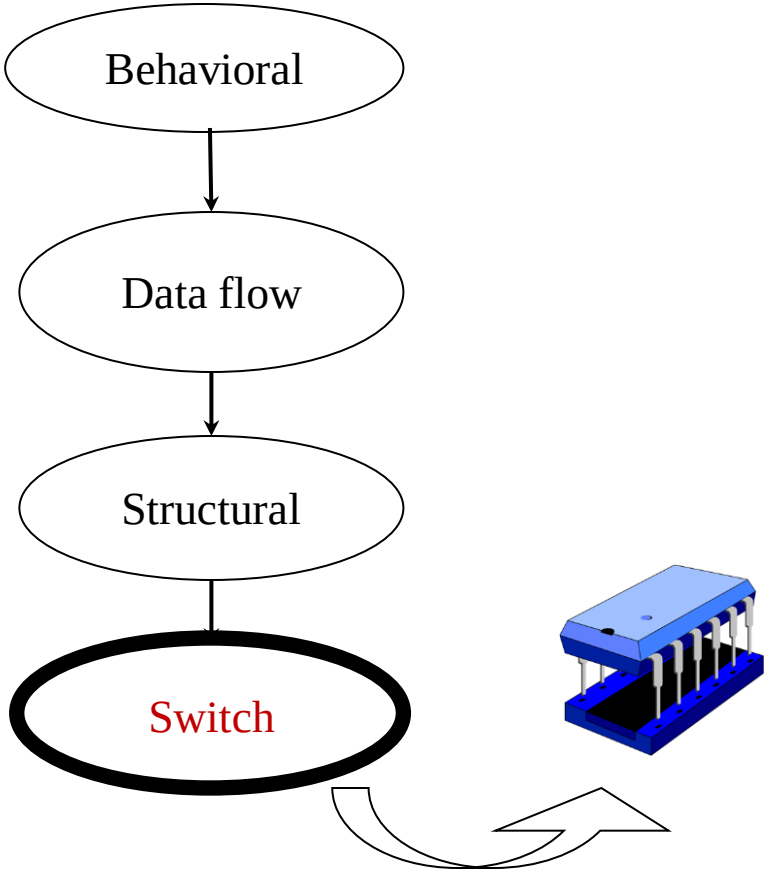


Levels of Abstraction

Also called as Gate level modeling. Module is designed in terms of logic gates/predefined verilog modules and interconnections between the gates/modules.



Levels of Abstraction



Low level of abstraction. Module is designed in terms of switches/transistors and hence the designer needs to know about the switch level implementation details.

Gate Level Modeling

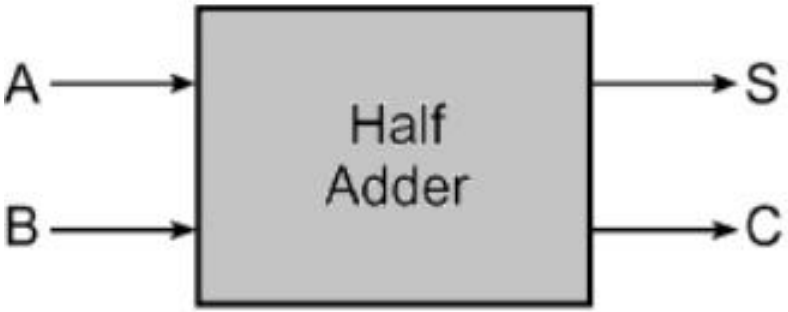
- A Gate level description is based on the structure of the logic circuit.
- A gate level description uses the verilog primitive gates to produce the desired output

Verilog Primitive Gates

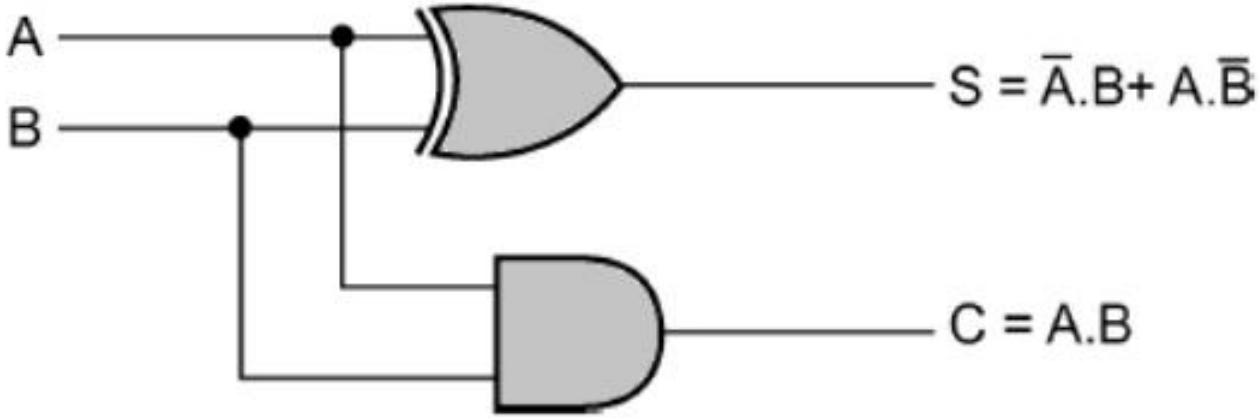
and(f,a,b,…)	f = (a · b …)
or(f,a,b,…)	f = (a + b + …)
not(f,a)	f = a'
nand(f,a,b,…)	f = (a · b …)'
nor(f,a,b,…)	f = (a + b + …)'
xor(f,a,b,…)	f = (a ⊕ b ⊕ …)
xnor(f,a,b,…)	f = (a ⊙ b ⊙ …)

- You can also “name” the gates if you like …
  - and And1(z1,x,y); // this gate is named And1

Half Adder

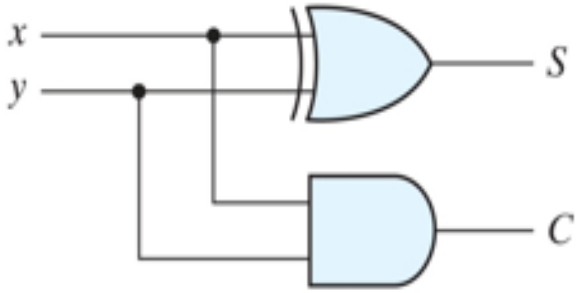


A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



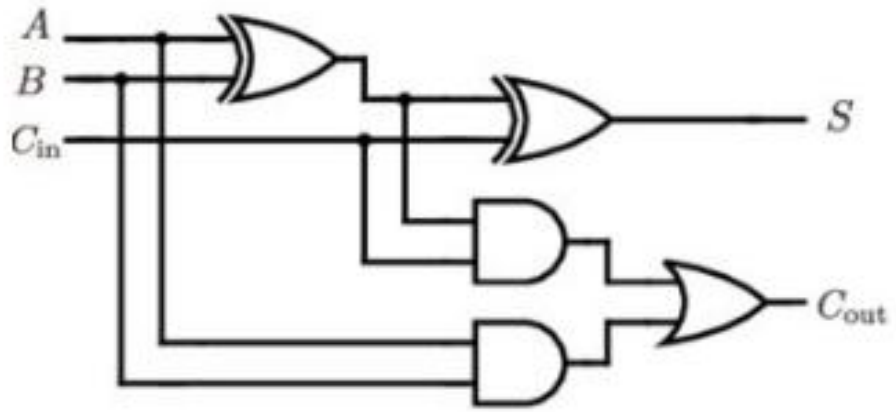
Gate Level Modeling – Half Adder

```
//Half Adder : Structural Verilog Description
module ha_gate_level (x,y,s,c);
  input x,y;
  output s,c;

  xor x1(s, x, y);
  and a1(c, x, y);

endmodule
```

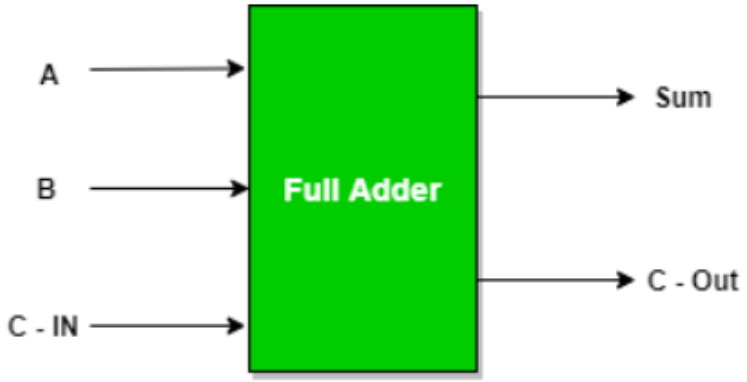


$S = x \oplus y$
$C = xy$

Full Adder



Inputs			Outputs	
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

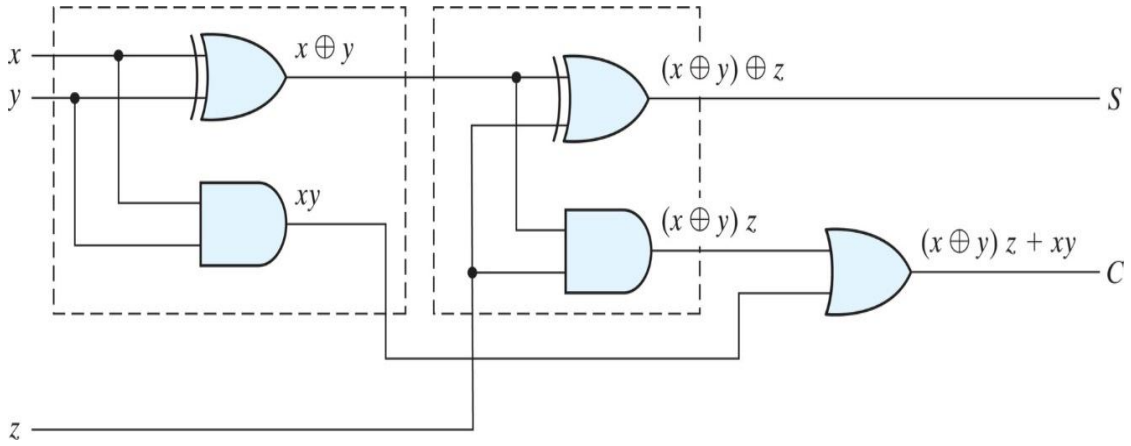


SUM : S = A'B'C+A'B C'+AB'C'+ABC (EX-OR GATE)

CARRY: C = AB+BC+CA

Gate Level Modeling - Full Adder

```
//Full Adder : Structural Description
module fa_gate_level(x,y,z,s,c);
  input x,y,z;
  output s,c;

  wire w1,w2,w3;

  xor xor1(w1,x,y);
  xor xor2(s,w1,z);
  and A1(w2, x,y);
  and A2(w3,w1,z);
  or O1(c, w3,w2);

endmodule
```



Dataflow Modeling

- A dataflow description is based on function rather than structure.
- A dataflow uses a number of operators that act on operands to produce the desired function
- Boolean equations are used in place of logic schematics.
- Continuous assignment(**assign**) is the most basic statement used in dataflow modeling

Dataflow Modeling – Half Adder

```
//Half Adder : Dataflow description
module ha_df (x,y,s,c);
  input x,y;
  output s,c;

  assign s = x^y;
  assign c = x & y;

  //assign {c,s} = x + y; //Alternate Method

endmodule
```

Operators

Types of Operators	Symbols
Arithmetic Operators	+, -, *, /, %
Relational Operators	<, <=, >, >=
Logical Equality Operators	==, !=
Case Equality Operators	===, !==
Logical Operators	!, &&, ||
Bit-wise Operators	~, &, |, ^, ~^
Unary Reduction Operators	&, ~&, |, ~|, ^, ~^
Shift Operators	<<, >>
Conditional Operators	? :
Concatenation Operators	{ }
Replication Operator	{ { } }

Continuous Assignment Examples

```
wire [3:0] X,Y,R;
wire [7:0] P;
wire r, a, cout, cin;
```

```
assign R = X | (Y & ~Z);
```
use of bit-wise Boolean operators

```
assign r = &X;
```
example reduction operator

```
assign R = (a == 1'b0) ? X : Y;
```
conditional operator

```
assign P = 8'hff;
```
example constants

```
assign P = X * Y;
```
arithmetic operators (use with care!)

```
assign P[7:0] = {4{X[3]}, X[3:0]};
```
(ex: sign-extension)

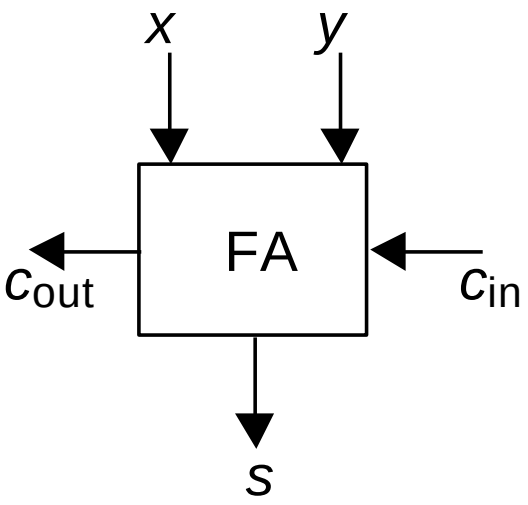
```
assign {cout, R} = X + Y + cin;
```
bit field concatenation

```
assign Y = A << 2;
```
bit shift operator

```
assign Y = {A[1], A[0], 1'b0, 1'b0};
```
equivalent bit shift

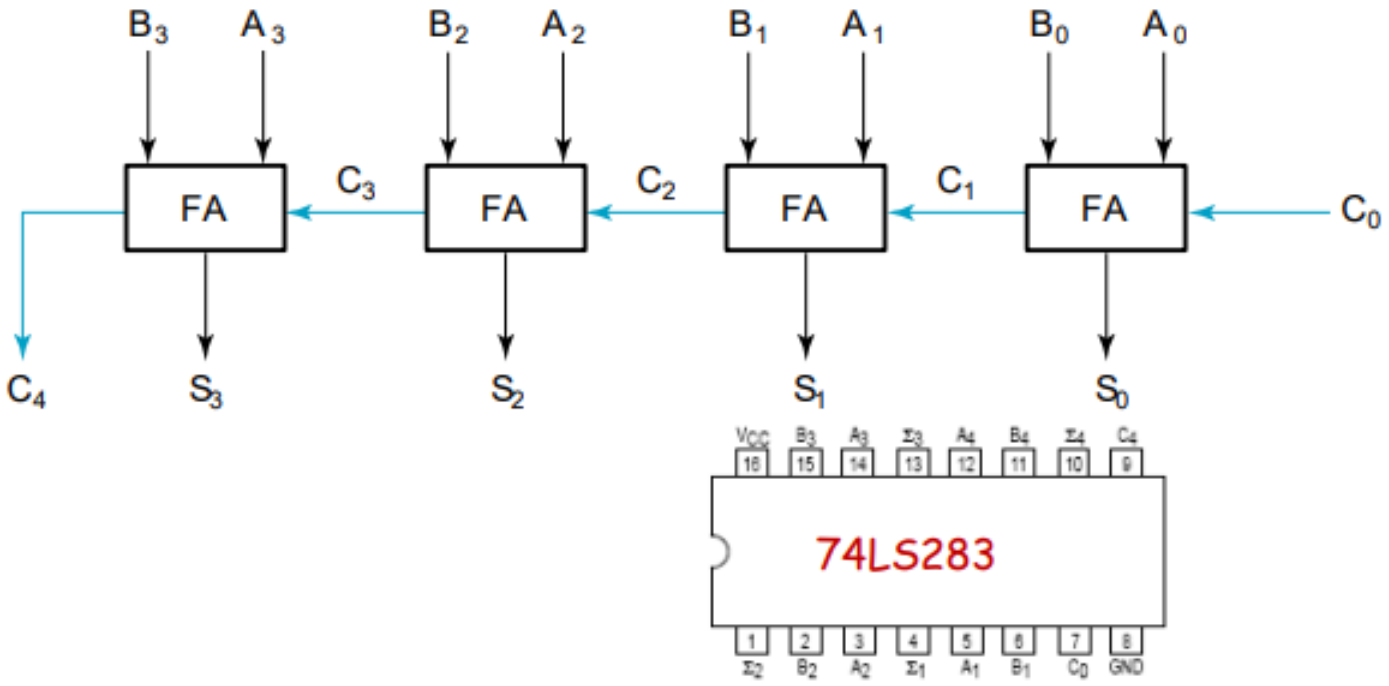
Ripple Carry Adder

Inputs			Outputs	
x	y	c_{in}	c_{out}	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



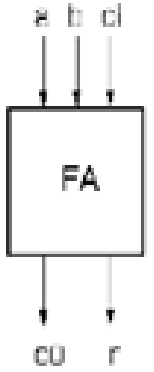
Four bit Ripple Carry Adder

✓ A four-bit Ripple Carry Adder made from four 1-bit Full Adders:

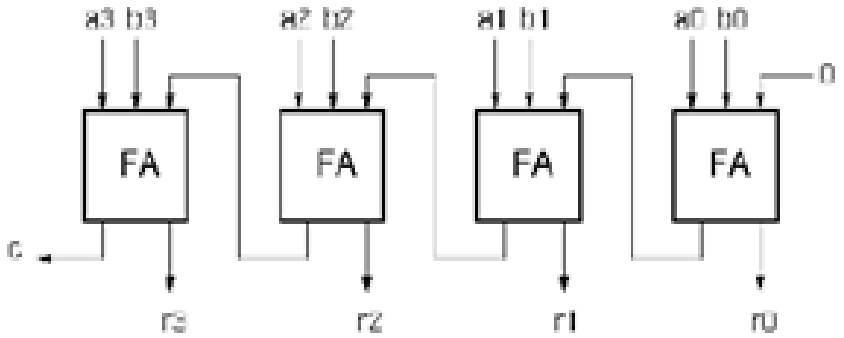


Ripple Carry Adder

```
module FullAdder(a, b, ci, r, co);
   input a, b, ci;
   output r, co;

   assign r = a ^ b ^ ci;
   assign co = a&ci + a&b + b&cin;

endmodule
```




```
module Adder(A, B, R);
   input [3:0] A;
   input [3:0] B;
   output [4:0] R;

   wire c1, c2, c3;
   FullAdder
     add0(.a(A[0]), .b(B[0]), .ci(1'b0), .co(c1),   .r(R[0]) ),
     add1(.a(A[1]), .b(B[1]), .ci(c1),   .co(c2),   .r(R[1]) ),
     add2(.a(A[2]), .b(B[2]), .ci(c2),   .co(c3),   .r(R[2]) ),
     add3(.a(A[3]), .b(B[3]), .ci(c3),   .co(R[4]), .r(R[3]) );
   endmodule
```



Behavioral Modeling

- Specifies how a particular design should respond to a given set of input.
- Behavioral modeling uses structured procedures to specify the behaviour of the circuit.
- May be specified by
    - Boolean expressions
    - Algorithms written in standard HLL like C

Ex : {Carry,Sum} = a+b+c;

Structured Procedures

- Two basic structured procedure statements or procedural blocks are
  - **always** (synthesizable)
  - **initial** (non-synthesizable)
- All behavioral statements can appear only inside these blocks
- Each always or initial block has a separate activity flow (concurrency)
- There can be multiple always and initial blocks in a module
- All procedural blocks start from simulation time 0
- Cannot be nested

Structured Procedures: `always` statement

- Starts at time 0
- Execute the statements in a looping fashion
- Multiple always blocks, execute in parallel
  All start at time 0

Syntax:

```
always @(sensitivity list)
begin
    // behavioral statements
end
```

Behavioral Modeling – Half Adder

```
//Half Adder : Behavioral description
module ha_beh (x,y,s,c);
  input x,y;
  output s,c;
  reg s, c;
  always @(a or b)
    begin
       s = x ^ y;
       c = x & y;
      //{c,s} = x + y;
    end

endmodule
```

Behavioral Modeling

"always" block example:

```
module and_or_gate (out, in1, in2, in3);
  input     in1, in2, in3;
  output    out;
  reg       out;

  always @(in1 or in2 or in3) begin
     out = (in1 & in2) | in3;
  end

endmodule
```

"reg" type declaration. Not really a register in this case. Just a Verilog rule.

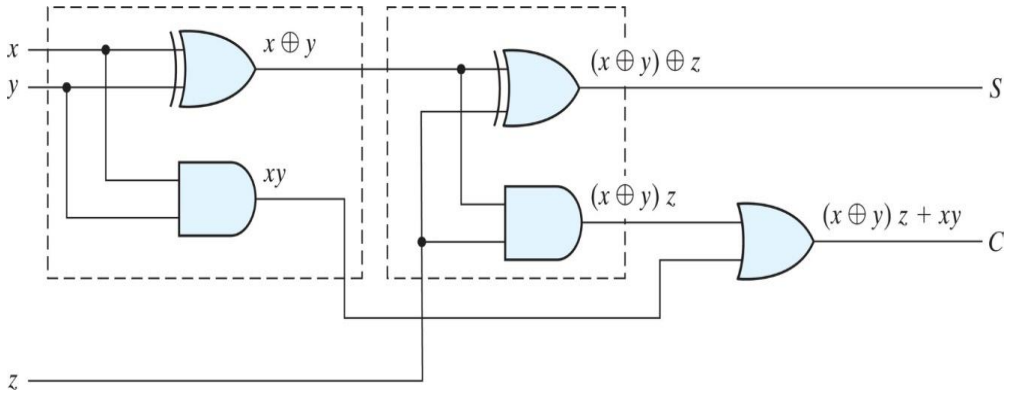
keyword

"sensitivity" list, triggers the action in the body.

brackets multiple statements (not necessary in this example.

Gate Level Modeling – Full Adder using Half adder

```
//Full Adder using 2 Half Adder : Structural Description
module fa_gate_level(x,y,z,s,c);
  input x,y,z;
  output s,c;

  wire s_ha1,c_ha1,c_ha2;

  //port mapping by order
  ha_gate_level HA1(x,y,s_ha1,c_ha1);
  //port mapping by name
  //ha_gate_level HA1( .a(a), .b(b), .s(s_ha1), .c(c_ha1) );

  ha_gate_level HA2(z, s_ha1, s, c_ha2);

  or O1(c, c_ha1, c_ha2);

endmodule
```




```
//Half Adder : Structural Verilog Description
module ha_gate_level (x,y,s,c);
  input x,y;
  output s,c;

  xor x1(s, x, y);
  and a1(c, x, y);

endmodule
```

Modelsim

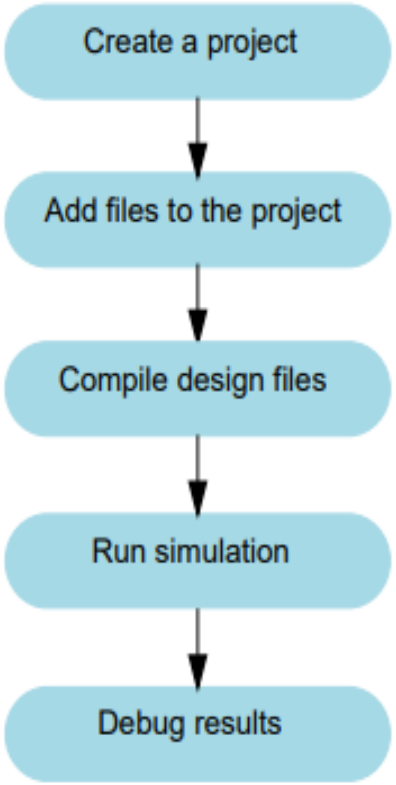
Modelsim

- ModelSim is a simulation verification and simulation tool by Siemens (previously developed by Mentor Graphics) for designs developed using the hardware description languages such as VHDL, Verilog, System Verilog
- ModelSim can be used independently, or in conjunction with Intel Quartus Prime.
- Simulation is performed using the graphical user interface (GUI), or automatically using scripts
- **Tool: https://tinyurl.com/modelsimtool**

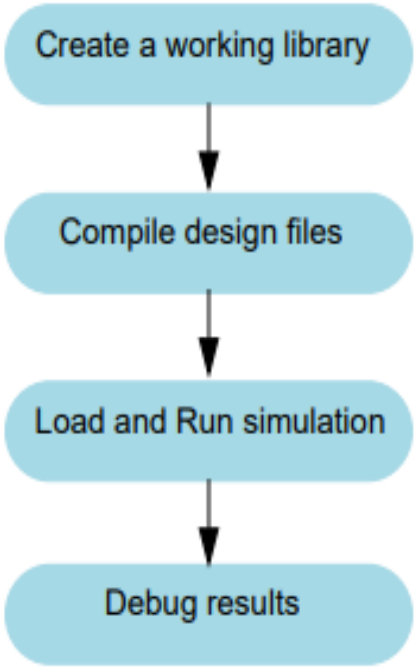
Simulation Flow

- Though you don't have to use projects in ModelSim, they may ease interaction with the tool and are useful for organizing files and specifying simulation settings

Method 1

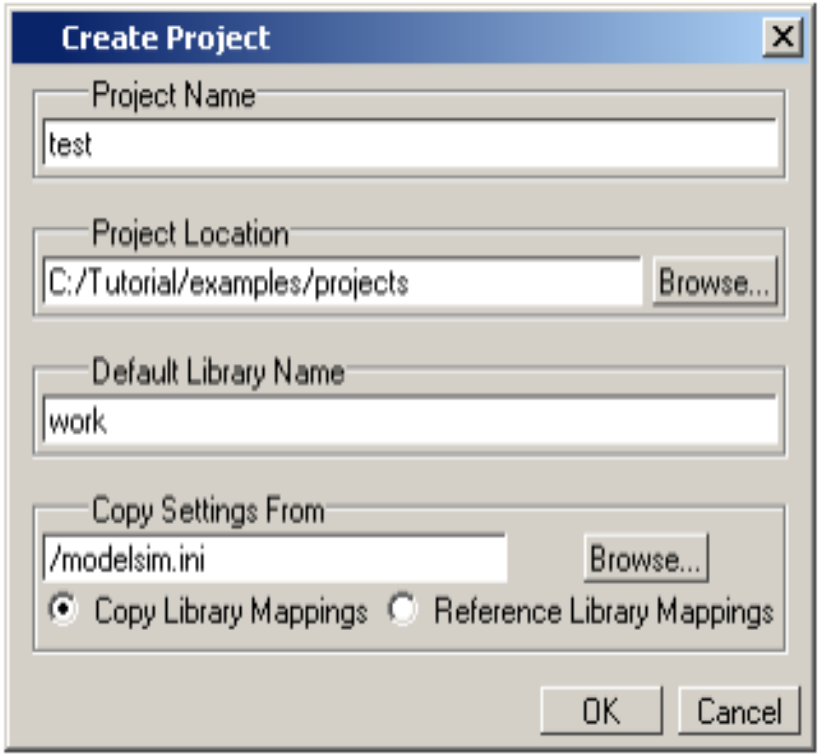


Alternative Method



Create a New Project

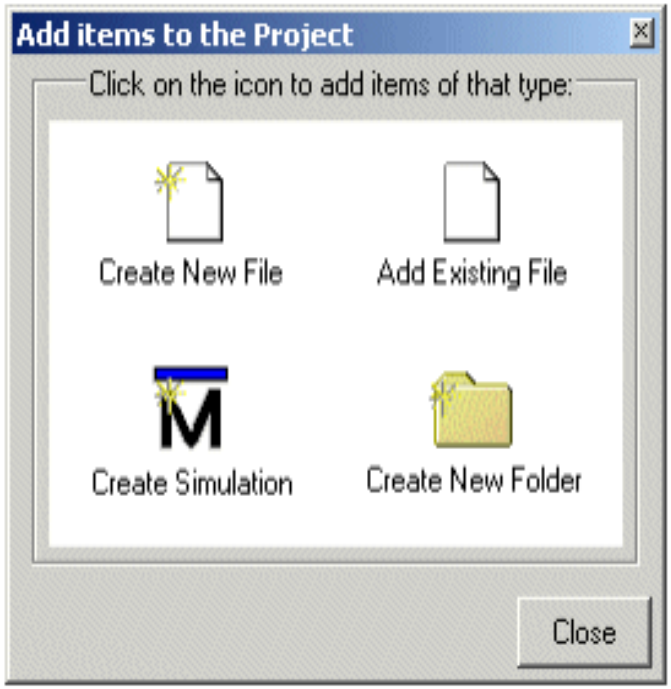
- Select **File > New > Project** (Main window) from the menu bar.\
- This opens the Create Project dialog where you can enter a Project Name, Project Location (i.e., directory), and Default Library Name



- Type the Project Name
- Click the Browse button for the Project Location field to select a directory where the project file will be stored
- Leave the Default Library Name set to work.
- Click OK.

Add Objects to the Project

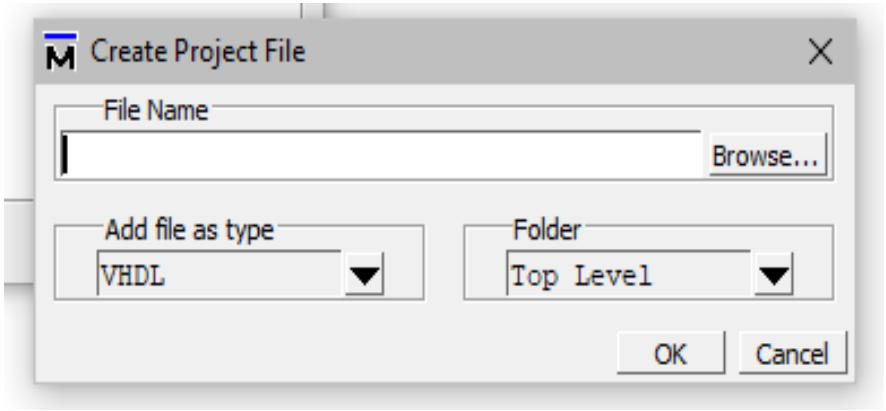
- Once you click OK to accept the new project settings, a blank Project window and the “Add items to the Project” dialog will appear



- Click Create New File if file has to be created or click Add Existing File if the file is available.

Create New File

- Once you click Create New File, Create Project File dialog will appear.

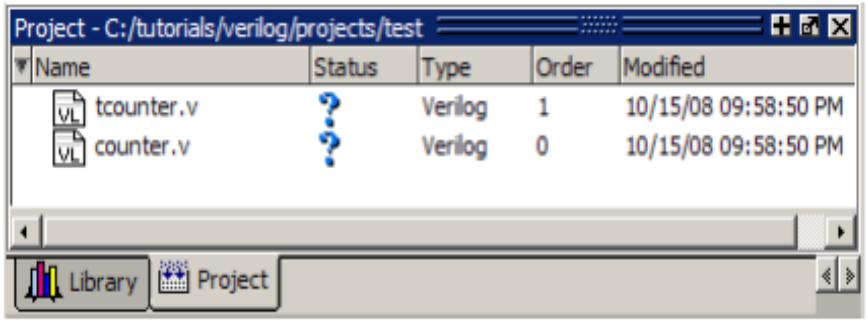


- Type the File name to be created
- Select Add file as type as **Verilog** from the drop down menu (By default it will be VHDL).
- Click OK.

- The new file will be added to the project. It can be selected and the design can be typed in the editor.

Compilation and Simulation

- New files show the status as ?



- Select the file, right click – compile – Compile selected.
- If compilation is successful, status changes to green colour tick mark.
- Go to work library, select the module to be simulated, right click and select simulate.
- Force the inputs and check the outputs in the wave window.

Reference

- Samir Palnitkar, “Verilog HDL: A Guide to Digital Design and Synthesis”, 2009, 2nd edition, Prentice Hall of India Pvt. Ltd.
- Stuart Sutherland, “Verilog® HDL Quick Reference Guide” based on the Verilog-2001 standard